BAHRIA UNIVERSITY, ISLAMABAD

Department of Software Engineering

PROJECT REPORT

Object-Oriented Programming (OOP)

Banking Management System

Qt Framework · C++17 · Microsoft SQL Server

Name	Enrollment / University Email
Muhammad Hammad Ateeq	09-131252-047 – 09-131252-047@student.bahria.edu.pk
Muhammad Shayan Chaudhary	09-131252-061 – 09-131252-061@student.bahria.edu.pk

GitHub Repository: github.com/Hammad4122/Banking-Management-System_QT_Application

1. Executive Summary

The Banking Management System (BMS) is a fully functional, desktop-based banking application built as the culminating project for the Object-Oriented Programming (OOP) course at Bahria University, Islamabad. The system is developed using C++17, the Qt 6 framework, and Microsoft SQL Server, and delivers a realistic simulation of core banking operations through a polished, professional graphical user interface.

The application demonstrates the practical application of fundamental and advanced OOP principles — including inheritance, encapsulation, polymorphism, and abstraction — alongside real-world software engineering practices such as layered architecture, secure password hashing, and event-driven programming.

Version 2 of the project extends the system with a fully implemented Admin Dashboard, role-based login, a dedicated AdminCredentials table, and a refactored DashboardWindow that serves as a shared shell for both user and admin sessions. Users can create accounts, securely log in, deposit and withdraw funds, transfer money between accounts, and review a live transaction history. Administrators can monitor system-wide statistics, browse all registered users and accounts, and manage admin credentials directly from the dashboard.

2. Introduction

2.1 Background & Motivation

Modern financial institutions rely on robust, scalable software systems to manage billions of transactions daily. While large-scale banking systems involve distributed architectures, microservices, and cloud infrastructure, the core logic — account management, authentication, fund transfers, and transaction logging — remains consistent across scales.

This project was conceived to bridge the gap between theoretical OOP concepts taught in the classroom and practical, industry-inspired software development. By building a complete banking system, the team gained hands-on experience with real database integration, UI/UX design, security fundamentals, and the software development lifecycle. Version 2 deepened this experience by introducing role-based access control, a multi-tab admin panel, and system-wide reporting.

2.2 Project Scope

The project encompasses the following functional domains:

- User registration and secure authentication with SHA-256 password hashing
- Account creation automatically linked to each registered user
- Core financial operations: Deposit, Withdrawal, and Fund Transfer
- Transaction history retrieval and display in a live, sortable table

- TPIN-based secondary authentication for every financial operation
- Real-time balance and financial summary (Income / Expense) display
- Dual-theme (Light Blue and Dark Emerald) UI theming system
- Automatic database schema initialization on first launch
- Role-based login: regular users and an admin role (NEW in v2)
- Admin Dashboard with system overview, user management, account browsing, and credential management (NEW in v2)

2.3 Project Objectives

1. Implement a fully operational banking desktop application using C++ and Qt 6.
2. Apply OOP design principles (inheritance, encapsulation, polymorphism) across the codebase.
3. Integrate a relational database (SQL Server via ODBC) for persistent data storage.
4. Implement SHA-256 password and TPIN hashing for secure credential management.
5. Deliver a professional, theme-aware UI with real-time data updates.
6. Practice software engineering patterns such as signal-slot communication and session management.
7. Implement role-based access control distinguishing regular users from administrators. (NEW in v2)
8. Provide an admin panel for system monitoring and credential management. (NEW in v2)

3. System Overview

3.1 Architecture Overview

The application follows a layered, component-based architecture with clear separation of concerns. Version 2 introduces role-based routing at the orchestration layer: LoginWindow now presents a role selector, and MainWindow dispatches the login to either the user dashboard or admin dashboard accordingly.

Layer	Component(s)	Description
Presentation	LoginWindow, SignupWindow, DashboardWindow, UserDashboardWindow, AdminDashboardWindow, TransactionDialog, TPinDialog	Handles all UI rendering, layout, and user interaction through Qt signals and slots.
Orchestration	MainWindow	QStackedWidget router; handles role dispatch on login (user vs. admin).
Business Logic	BasePage, UserSessionHandler	Shared theming, DB lifecycle; in-memory session state for authenticated user.

Data Access	BankDB (database.h / database.cpp)	All SQL Server communication via Qt SQL / ODBC. Admin queries and credential management added in v2.
Database	Microsoft SQL Server	Four normalized tables: Users, Accounts, Transactions, AdminCredentials (new in v2). Schema auto-created on first run.

3.2 Application Flow

On launch, MainWindow initializes a QStackedWidget with three pages: LoginWindow (index 0), SignupWindow (index 1), and DashboardWindow (index 2). The application starts at the login page.

LoginWindow now includes a role selector (QComboBox) offering "Login as User" and "Login as Admin". Depending on the selected role, the login button triggers one of two paths:

- User path — credentials are verified via BankDB::loginUser(); on success, a UserSessionHandler* is constructed and emitted with loginSuccessful(session). MainWindow calls DashboardWindow::initializeUserDashboard(session), which loads UserDashboardWindow into the content stack.
- Admin path — credentials are verified via BankDB::authenticateAdmin(user, pass); on success, adminLoginSuccessful() is emitted. MainWindow calls DashboardWindow::initializeAdminDashboard(), which loads AdminDashboardWindow into the content stack.

Logout in both paths resets the DashboardWindow, clears the session, and returns to the login page.

3.3 Technology Stack

Technology	Details
Language	C++17 (ISO standard, modern features)
UI Framework	Qt 6 (Qt Widgets, Qt SQL, Qt GUI, Qt Core)
Build System	CMake 3.16+ with AUTOUIC / AUTOMOC / AUTORCC
Database	Microsoft SQL Server (QODBC driver)
Hashing	SHA-256 via QCryptographicHash
Styling	Qt Style Sheets (QSS)
IDE	Qt Creator 13
Version Control	Git / GitHub

4. Object-Oriented Design

4.1 Class Hierarchy

The project implements a clean OOP class hierarchy centered on the BasePage base class. Version 2 adds three new classes: UserDashboardWindow (refactored from the original DashboardWindow content), AdminDashboardWindow, and a restructured DashboardWindow that acts as a shared shell.

4.2 Class Descriptions

4.2.1 BasePage (basepage.h / basepage.cpp)

BasePage is the abstract base class for all three application windows (LoginWindow, SignupWindow, DashboardWindow). It inherits from QWidget and enforces the following responsibilities:

- Owns a shared BankDB instance (db), ensuring a single database connection is maintained per window.
- Declares static isDarkMode flag and lightBlueTheme / darkEmeraldTheme QSS strings, making theme state global across all child windows.
- Provides applyCurrentTheme() and updateIcons() virtual-style methods, called by MainWindow after toggling the theme flag.
- Implements paintEvent() override to support QSS background-color rendering on plain QWidget subclasses.

4.2.2 LoginWindow (loginwindow.h / loginwindow.cpp)

Inherits from BasePage. Provides the authentication entry point. In v2, a role selector QComboBox ("Login as User" / "Login as Admin") was added directly below the password field. Key responsibilities:

- Validates fields and dispatches to BankDB::loginUser() for user credentials or BankDB::authenticateAdmin() for admin credentials.
- Hides the Signup link when the Admin role is selected.
- Emits loginSuccessful(session) (user path) or adminLoginSuccessful() (admin path) to MainWindow.
- resetForm() clears all fields and hides error labels on logout.

4.2.3 SignupWindow (signupwindow.h / signupwindow.cpp)

Inherits from BasePage. Manages new user registration with comprehensive inline validation across all fields: First Name, Last Name, Username, Email, Mobile Number, Password, Confirm Password, and TPIN.

4.2.4 DashboardWindow (dashboardwindow.h / dashboardwindow.cpp)

Inherits from BasePage. Refactored in v2 to act as a shared shell rather than directly rendering user content. Its internal QStackedWidget holds either a UserDashboardWindow or an AdminDashboardWindow. Key methods added in v2:

- initializeUserDashboard(session) — injects the user session into UserDashboardWindow and switches the stack to it.
- initializeAdminDashboard() — calls AdminDashboardWindow::loadData() and switches the stack to the admin panel.
- The header bar (application title, logout button, theme toggle, settings icon, avatar) remains in DashboardWindow and is shared by both roles.

4.2.5 UserDashboardWindow (userdashboardwindow.h / userdashboardwindow.cpp) — NEW in v2

A QWidget subclass that houses all user-facing dashboard content previously implemented directly in DashboardWindow. Responsibilities:

- loadSession(session) — populates the debit card widget, income/expense summary cards, and transaction table from the UserSessionHandler.
- Deposit, Transfer, and Withdraw buttons each launch a modal TransactionDialog.
- refreshBalanceAndTable() slot refreshes all financial data after a transaction completes.

4.2.6 AdminDashboardWindow (admindashboardwindow.h / admindashboardwindow.cpp) — NEW in v2

A QWidget subclass implementing a tabbed admin control panel. It contains four tabs:

- Overview tab — displays four styled stat cards: Total Users, Total System Balance, Total Deposits (All), and Total Withdrawals. Data is fetched from BankDB via aggregate SQL queries.
- Users tab — a searchable QTableView listing all registered users (user_id, username, email, mobile, created_at). A QLineEdit search field filters rows in real time via onSearchUsers().
- Accounts tab — a QTableView listing all accounts with their current balances.
- Credentials tab — allows the admin to change the admin username and password. The current password must be verified before any change is accepted, and the new credentials are SHA-256 hashed before storage.

4.2.7 BankDB (database.h / database.cpp)

The Data Access Object (DAO) class. All SQL operations are centralized here. New in v2:

- initializeSchema() now also creates an AdminCredentials table and inserts a default admin record (username: admin, password: admin) if none exists.
- authenticateAdmin(username, password) — verifies admin credentials against the AdminCredentials table.
- changeAdminCredentials(currentPassword, newUsername, newPassword) — verifies the current password before updating both the admin username and password hash.
- System-wide aggregate queries: getTotalUsers(), getTotalSystemBalance(), getTotalDeposits(), getTotalWithdrawals() — used by the admin Overview tab.

- getAllUsers(filter) and getAllAccounts() — return full data sets for the admin Users and Accounts tabs.

4.2.8 UserSessionHandler (usersessionhandler.h / usersessionhandler.cpp)

Unchanged in structure from v1. A plain C++ class (no Qt parent) acting as an in-memory session object for the authenticated user. Stores userId, firstName, lastName, username, email, mobileNo, accountId, balance, totalIncome, and totalExpense. Provides const-reference getters and setBalance() for post-transaction refreshes.

4.2.9 TransactionDialog (transactiondialog.h / transactiondialog.cpp)

A QDialog subclass serving all three transaction types (Deposit, Withdraw, Transfer) via a TransactionType enum. Dynamically shows or hides the receiver Account Number field for transfers. Validates all inputs, launches TPinDialog for secondary authentication, plays a success GIF animation, and emits updateBalance() on completion.

4.2.10 TPinDialog (tpindialog.h / tpindialog.cpp)

A minimal QDialog prompting for the 4-digit Transaction PIN. Features password echo mode, input validation (Confirm button disabled until exactly 4 digits are entered), and theme-aware styling.

4.3 OOP Principles Applied

Principle	Implementation in Project
Inheritance	LoginWindow, SignupWindow, and DashboardWindow all inherit from BasePage. UserDashboardWindow and AdminDashboardWindow inherit from QWidget and are composed inside DashboardWindow.
Encapsulation	BankDB hides all SQL internals behind a clean public API. UserSessionHandler encapsulates user data with private members and public getters/setters. AdminDashboardWindow encapsulates all admin query logic and tab-building internally.
Abstraction	BasePage abstracts the theme system and database lifecycle. DashboardWindow abstracts the shell UI. TransactionDialog abstracts three transaction types behind a single reusable dialog.
Polymorphism	applyCurrentTheme() and updateIcons() are called on all BasePage subclasses from MainWindow::handleTheme(). Each subclass applies theming to its own widget tree.
Signal & Slot	loginPage emits loginSuccessful(session) or adminLoginSuccessful(). MainWindow receives both and routes to the appropriate dashboard initializer. AdminDashboardWindow also emits themeChangeRequested() forwarded to the shell.
RAII	MainWindow::~MainWindow() calls delete currentSession to prevent memory leaks. TransactionDialog uses Qt::WA_DeleteOnClose to auto-delete dialogs.

Composition	DashboardWindow composes UserDashboardWindow and AdminDashboardWindow via a QStackedWidget. AdminDashboardWindow composes its four tab widgets internally. This replaces the deep inheritance that would otherwise be needed for role-specific views.

5. Database Design

5.1 Schema Overview

The system uses Microsoft SQL Server with four normalized relational tables. The schema is automatically initialized by initializeSchema() on the first database connection. Version 2 adds the AdminCredentials table to support role-based authentication and secure admin credential management.

5.2 Table Definitions

5.2.1 Users Table

Column	Type & Description
user_id	INT IDENTITY(1,1) — Primary Key, auto-incremented
first_name	NVARCHAR(30) — NOT NULL
last_name	NVARCHAR(30) — NOT NULL
user_name	NVARCHAR(15) — Unique, NOT NULL
email	NVARCHAR(100) — Unique, NOT NULL
password	NVARCHAR(64) — SHA-256 hashed password (hex string)
mobile_no	NVARCHAR(11) — Unique, NOT NULL
tpin	NVARCHAR(64) — SHA-256 hashed Transaction PIN
created_at	DATETIME — Account creation timestamp, DEFAULT GETDATE()

5.2.2 Accounts Table

Column	Type & Description
account_id	INT IDENTITY(1000,1) — Primary Key, starts at 1000
user_id	INT (FK) — References Users(user_id) ON DELETE CASCADE
balance	DECIMAL(18,2) — Current account balance, DEFAULT 0.00
currency	NVARCHAR(10) — Currency code, DEFAULT 'PKR'

5.2.3 Transactions Table

Column	Type & Description
transaction_id	INT IDENTITY(1,1) — Primary Key, auto-incremented
account_id	INT (FK) — References Accounts(account_id) ON DELETE CASCADE
transaction_type	NVARCHAR(20) — 'Deposit', 'Withdrawal', or 'Transfer'
amount	DECIMAL(18,2) — Transaction amount, NOT NULL
balance_after	DECIMAL(18,2) — Account balance after the transaction
remarks	NVARCHAR(255) — User-supplied transaction notes
transaction_date	DATETIME — Timestamp, DEFAULT GETDATE()

5.2.4 AdminCredentials Table — NEW in v2

A new single-row table that stores the admin account credentials separately from the Users table, keeping admin authentication isolated from the user authentication flow.

Column	Type & Description
id	INT PRIMARY KEY — Always 1 (single admin record)
admin_username	NVARCHAR(50) NOT NULL — Admin login username
password_hash	NVARCHAR(64) NOT NULL — SHA-256 hashed admin password

Default credentials inserted on first run: username admin, password admin (SHA-256 hashed). These should be changed immediately via the Credentials tab in the Admin Dashboard.

6. User Interface

The application's UI is built entirely with Qt Widgets and styled using Qt Style Sheets (QSS). The design follows a modern banking aesthetic with two complete themes. Version 2 introduces the Admin Dashboard UI and a role selector on the Login page.

6.1 Login Window

The Login Window presents a centered card on a light blue (or dark) background. It includes an application header with logo and theme toggle, a username field, a password field (masked), a role selector (QComboBox offering User and Admin), an error status label, a Login button, and a navigation link to Signup. The Signup link is hidden automatically when the Admin role is selected, since admins cannot self-register.

6.2 Signup Window

The Signup Window mirrors the Login layout with a wider card (550x500 px) and additional fields. Inline validation feedback appears in real time below each field group as the user interacts with the form. Fields include: First Name, Last Name, Username, Email, Mobile Number, Password, Confirm Password, and TPIN.

6.3 User Dashboard Window

The user dashboard is rendered by UserDashboardWindow, which is loaded into DashboardWindow's content stack after a successful user login. It consists of:

- Debit Card Widget — gradient blue/green card displaying balance, card holder name, masked card number, and Mastercard logo.
- Income & Expense Cards — aggregate totals from all Deposit and Withdrawal transactions.
- Quick Action Buttons — Deposit, Transfer, Withdraw, each launching a modal TransactionDialog.
- Transaction History Table — live QTableView showing Date, Type, Amount, and Remarks, sorted newest-first.

6.4 Admin Dashboard Window — NEW in v2

The Admin Dashboard is rendered by AdminDashboardWindow, loaded into DashboardWindow's content stack after a successful admin login. It is organized into four tabs:

Overview Tab

Displays four styled stat cards arranged in a 2x2 grid:

- Total Users — count of all registered users (blue accent).
- Total System Balance — sum of all account balances across the system (green accent).
- Total Deposits — aggregate of all deposit transactions system-wide (orange accent).
- Total Withdrawals — aggregate of all withdrawal transactions system-wide (red accent).

Users Tab

A searchable QTableView listing all registered users. A QLineEdit search field at the top of the tab filters the table in real time by username, email, or other user fields. The table is populated via BankDB::getAllUsers() and refreshed whenever the tab is activated.

Accounts Tab

A QTableView listing all accounts with their account IDs, linked user IDs, current balances, and currency. Refreshed whenever the tab is activated via BankDB::getAllAccounts().

Credentials Tab

Allows the admin to update the admin username and password. The form requires the current password as verification before saving any changes. On submission, BankDB::changeAdminCredentials() verifies the current password hash, then updates the stored username and new password hash. A status label provides immediate feedback (success or error) after the operation.

6.5 Transaction Dialog

The Transaction Dialog is a modal QDialog that adapts its layout based on the selected transaction type. For transfers, a receiver Account Number field is shown. All transactions require a TPIN before execution. On success, an animated GIF is displayed for 2.1 seconds before the dialog closes.

7. Security Design

7.1 Password Hashing

All passwords (user and admin) are hashed with SHA-256 using Qt's built-in QCryptographicHash class before storage. The raw password is never written to the database. On login, the entered password is hashed with the same algorithm and compared to the stored hash.

7.2 Transaction PIN (TPIN)

Every financial operation (Deposit, Withdrawal, Transfer) requires the user to enter a 4-digit TPIN. This TPIN is stored as a separate SHA-256 hash in the Users table, independent of the account password. It provides a second layer of authentication, simulating the PIN-at-terminal model used in real banking.

7.3 Admin Credential Security — NEW in v2

Admin credentials are stored in a dedicated AdminCredentials table, isolated from the Users table. The admin password is SHA-256 hashed. The Credentials tab requires the current password before allowing any change, preventing unauthorized credential updates even if the admin session is left unattended. Default credentials (admin / admin) are inserted on first run and should be changed immediately.

7.4 SQL Injection Prevention

All database queries use Qt's prepared statement API (QSqlQuery::prepare() with addBindValue()). User-supplied input is never concatenated into SQL strings, completely eliminating the risk of SQL injection attacks.

7.5 Input Validation

Client-side validators are applied to all input fields: QDoubleValidator limits the amount field; QRegularExpressionValidator enforces 4-digit numeric format for TPIN; email and mobile number uniqueness are verified against the database before registration.

8. Features Summary

Feature	Description
Secure Registration	Full form validation, unique constraint enforcement, SHA-256 password and TPIN hashing.
Authenticated Login	Username + hashed password verification with clear error messaging and role selection.
Role-Based Access (NEW)	Login page offers User and Admin roles. Each role loads a different dashboard content widget.
Live User Dashboard	Real-time balance, income, expense, and transaction table on a single screen.
Deposit	Add funds to account with remarks; balance updates immediately in the UI.
Withdrawal	Withdraw funds with balance sufficiency check and real-time UI refresh.
Fund Transfer	Transfer to any existing account; prevents self-transfer; verifies recipient account.
TPIN Authentication	4-digit PIN required before every financial transaction.
Transaction History	Sortable, read-only table linked directly to QSqlQueryModel.
Admin Overview (NEW)	Stat cards showing total users, system balance, total deposits, and total withdrawals.
Admin User Browser (NEW)	Searchable table of all registered users, filterable in real time.
Admin Account Browser (NEW)	Table of all accounts with balances, refreshed on tab activation.
Admin Credentials Mgmt (NEW)	Change admin username and password from the dashboard; current password verification required.
Dual Theming	Light Blue and Dark Emerald themes toggle globally with a single button.
Auto Schema Init	All four database tables created automatically if they do not exist on first launch.
Success Animation	Animated GIF plays on successful transaction for positive UX feedback.
Cross-Platform DB	ODBC connection string adapts for Windows (Windows Auth) and Linux (SQL Auth).

9. Setup & Installation Guide

9.1 Prerequisites

Requirement	Details
Qt Framework	Qt 6.x (Widgets, SQL, GUI, Core modules) — available at qt.io/download
C++ Compiler	MinGW 64-bit (Windows) or GCC (Linux)
CMake	Version 3.16 or higher
SQL Server	Microsoft SQL Server (Express edition acceptable). Database name: BankDB
ODBC Driver	SQL Server ODBC Driver — bundled on Windows; ODBC Driver 18 required on Linux
Qt Creator	Recommended IDE for building the project (version 13+)

9.2 Database Configuration

1. Install Microsoft SQL Server and create a database named BankDB.
2. On Windows: ensure SQL Server is accessible at localhost\SQLEXPRESS with Windows Authentication.
3. On Linux: configure a remote SQL Server instance with SQL Authentication (UID: sa).
4. No manual table creation is required — the application creates all four tables on first run, including the AdminCredentials table with default credentials (admin / admin).

9.3 Build Steps

5. Clone the repository: git clone https://github.com/Hammad4122/Banking-Management-System_QT_Application.git
6. Open Qt Creator → File → Open File or Project → select CMakeLists.txt.
7. Configure the project kit (Desktop Qt 6.x MinGW or GCC).
8. Build the project using the Build menu (Ctrl+B).
9. Run the application (Ctrl+R). The application will connect to SQL Server and initialize the schema automatically.
10. On first login, use admin / admin for the admin role, then immediately update credentials via the Credentials tab.

10. Testing & Validation

The application was tested manually through structured scenario-based testing covering all functional paths, including the new admin features added in v2.

Test Scenario	Expected Result	Status
Register with all valid fields	Account created, redirected to login	✓ Pass
Register with duplicate username	Inline error message shown	✓ Pass
Login (User role) with correct credentials	User dashboard loads with user data	✓ Pass
Login (User role) with wrong password	Error label displayed	✓ Pass
Login (Admin role) with correct credentials	Admin dashboard loads with all four tabs	✓ Pass
Login (Admin role) with wrong credentials	'Invalid admin credentials' error shown	✓ Pass
Admin Overview tab data	Stat cards show correct aggregated values	✓ Pass
Admin Users tab search	Table filters in real time as text is typed	✓ Pass
Admin Accounts tab	All accounts and balances displayed correctly	✓ Pass
Admin change credentials (correct current pass)	Credentials updated; new login works	✓ Pass
Admin change credentials (wrong current pass)	Error label; no change made	✓ Pass
Deposit funds with valid TPIN	Balance updates, history refreshes	✓ Pass
Withdraw more than balance	'Not Enough Balance' shown	✓ Pass
Transfer to non-existent account	'Account does not exist' shown	✓ Pass
Enter wrong TPIN	'Wrong TPIN' error shown	✓ Pass
Toggle theme on all pages	Theme applies globally and consistently	✓ Pass
Logout and re-login (both roles)	Form resets, correct dashboard loads for each role	✓ Pass

11. Limitations & Future Work

11.1 Current Limitations

- Settings Window: The settingwindow class is declared but not yet implemented.
- No Multi-Account Support: Each user is linked to exactly one account.

- Hardcoded Database Credentials: The Linux connection string includes a hardcoded password (BMS@2026); this should be moved to a configuration file.
- No Session Timeout: There is no idle timeout mechanism to automatically log out inactive users.
- Card Number is Static: The displayed card number and expiry are hardcoded placeholders.
- No Email Notifications: Transaction confirmation emails are not sent.
- Single Admin Account: The AdminCredentials table stores a single admin record. Multi-admin support would require schema changes.
- No SQL Transaction Wrapping: The transfer operation performs two sequential SQL statements without a BEGIN TRANSACTION block; a network failure mid-transfer could leave balances inconsistent.

11.2 Proposed Future Enhancements

- Settings Window: Allow users to change passwords, update personal information, and configure notification preferences.
- Multi-Account Support: Enable users to open multiple account types (Savings, Current, Fixed Deposit).
- Transaction Receipt Export: Generate PDF receipts for completed transactions.
- Email / SMS Notifications: Integrate SMTP or a notification API for transaction alerts.
- Charts & Analytics: Visual spending breakdowns on the admin Overview tab using Qt Charts.
- Loan Management Module: Allow users to apply for and track loan repayments.
- Environment-based Configuration: Move connection strings to a .env or config.ini file.
- SQL Transaction Wrapping: Wrap fund-transfer operations in BEGIN TRANSACTION / COMMIT / ROLLBACK blocks for data integrity.
- Multi-Admin Support: Extend AdminCredentials to support multiple admin users with individual usernames and role levels.

12. Conclusion

The Banking Management System v2 successfully demonstrates the power of Object-Oriented Programming when applied to a real-world domain. By leveraging C++17, the Qt 6 framework, and Microsoft SQL Server, the team built a production-quality desktop application that implements secure authentication, real-time financial operations, a professional UI with dual theming, and a normalized relational database schema — all within a clean, maintainable class hierarchy.

Version 2 meaningfully extends the system with role-based access control, a full Admin Dashboard (Overview, Users, Accounts, and Credentials tabs), and a refactored architecture where the DashboardWindow acts as a shared shell composed with role-specific content widgets. The AdminCredentials table and associated BankDB methods provide a complete, secure credential lifecycle for the admin account.

The project reinforced key OOP concepts — inheritance through the BasePage hierarchy, encapsulation in BankDB and UserSessionHandler, polymorphism in the theme system, abstraction in the TransactionDialog, and composition in the DashboardWindow/AdminDashboardWindow relationship — in a practical, meaningful context far beyond traditional console-based exercises.

This project stands as a strong portfolio piece demonstrating our ability to design, build, and deliver complete software systems — from architecture and database design through to UI polish, security implementation, and role-based administration.

End of Report

github.com/Hammad4122/Banking-Management-System_QT_Application